

Email Spam Detection using Machine Learning Algorithms

Badal Singh Computer Science & Engineering Department, Oriental College of Technology Bhopal, Madhya Pradesh, India singh.badal3375@gmail.com	Rishabh Mishra Computer Science & Engineering Department Oriental College of Technology Bhopal, Madhya Pradesh, India rishabhmishra2855@gmail.com
---	---

Abstract:

Digital messages can be sent over the internet using an email (electronic mail). With email addresses, it enables users to compose, send, and receive messages instantly. Frequently, attachments such as documents, photos, or links are included. It is frequently used for both professional and personal communication. They frequently contain harmful links, advertisements, or phoney offers. Use spam filters, delete shady emails, and avoid clicking on unknown links to keep yourself safe. The goal of this research is to create machine learning models for email spam detection so that the system can reliably differentiate between spam and legitimate emails. A machine learning model that uses Naive Bayes, XG Boost Decision Tree Classifier, Random Forest, and SVM to detect new email classifications was developed using the Import spam dataset. The NLP technique was used to analyse and carry out data preprocessing, such as eliminating HTML tags, lowercase letters, stop words, handling emojis, and the email text's URL. The outcomes are contrasted with those of earlier email spam detection models. Using Naive Bayes (97.21), Random Forest (97.39), and XG Boost (98.12), the suggested machine learning method obtained the highest accuracy rate, 99.20. All implementations make use of Python and numerous libraries.

Keywords— Email spam detection, Machine Learning, XG Boost, SVM, Random Forest, voice text, Naive Bayes.

I. INTRODUCTION

In previous years, most spam filtering relied on manually blocking messages from specific email addresses. Machine learning is used in modern methods to detect spam more efficiently. Text analysis, domain blacklists and whitelists, and community-based approaches are examples of common strategies. One of the most popular methods for detecting spam is text analysis of email content. Among the most popular techniques used in these systems are algorithms like Naïve Bayes. However, using content analysis alone may result in false positives, which are emails that are legitimate but are mistakenly classified as spam.

Due to its widespread use in daily life, email has become a popular target for bad actors. In order to send phishing emails, fraudulent offers, dangerous links, and misleading messages intended to fool recipients, attackers frequently create fictitious accounts. These emails could be financial scams, requests for private information, or malware. This can be resolved by creating a machine learning model that can identify if an email is ham (legitimate) or spam. Classifying emails into various categories, such as spam, ham, promotions, and updates, is crucial because attackers commonly send out large volumes of messages with claims like winning rewards or prizes (e.g., \$50,000).

This system lowers the risk of online scams by recommending whether an email is harmful or harmless based on machine learning or deep learning models. This spam detection model is an effective software tool for spotting questionable emails because it drastically lowers the scam rate when compared to prior years. The model achieves high

accuracy rates of up to 99.20% by utilising both machine learning and deep learning techniques, along with natural language processing (NLP) for preprocessing. One of the oldest techniques, the blacklist approach, prevents emails from addresses or domains that have been specifically blacklisted. However, this strategy is no longer as successful because spammers are always coming up with new domains. On the other hand, emails from reliable domains or addresses are accepted by the whitelist technique, which filters all other emails into a low-priority queue.

Only after the sender replies to an authentication request sent by the filtering system are these sent.

Spam And Ham :

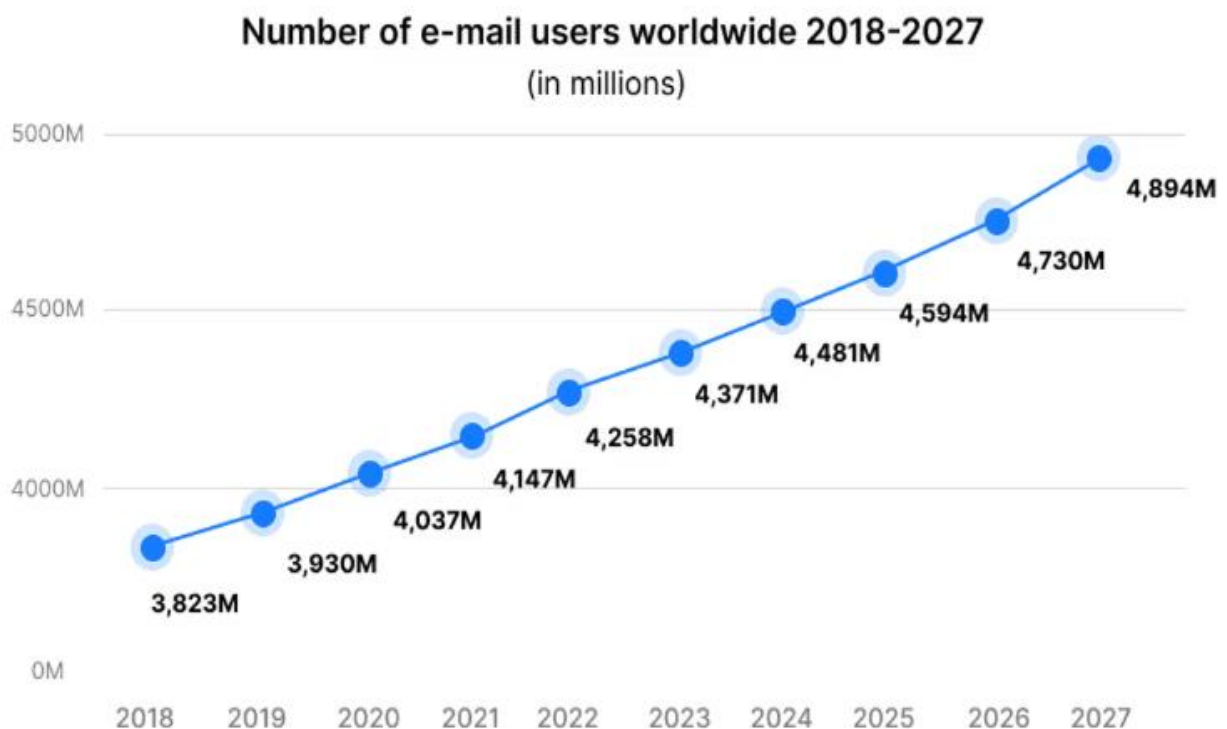


Fig 1: Diagram showing the increment in spam messages

The Fig.1[17] displays, in millions, the increase in email users worldwide between 2018 and 2027. One of the most popular digital communication tools for professional, academic, and

personal use is still email. The number of users was approximately 3,823 million in 2018, and it grew annually. In 2019 and 2020, the number increased to 3,930 million and 4,037 million, respectively. With 4,147 million users in 2021 and 4,258 million in 2022, this increasing trend persisted. The number rose to 4,371 million by 2023 and then to 4,481 million in 2024.

With 4,594 million users in 2025 and 4,730 million in 2026, the projected data also demonstrates continuous growth. It is anticipated that there will be 4,894 million email users worldwide by 2027. Overall, the graph shows that email is still the most popular and dependable form of communication in the world. The number of people using email is still rising steadily, despite the growing popularity of social media and instant messaging. This increasing trend suggests that email will continue to be an essential tool for international communication in the upcoming years due to growing internet usage, digital services, and business communication requirements.

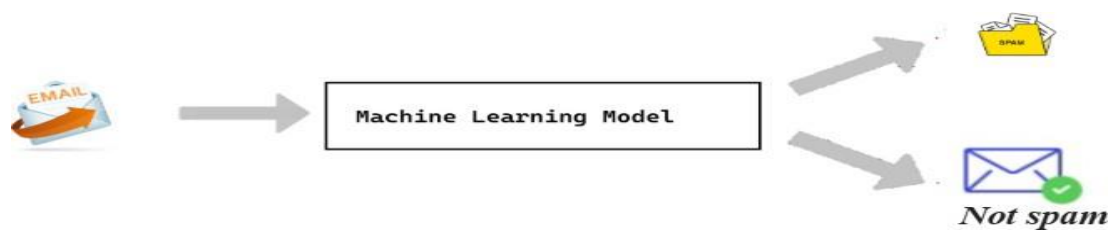


Fig 2: Classification into Spam and Ham

II: Research Objective and Significance

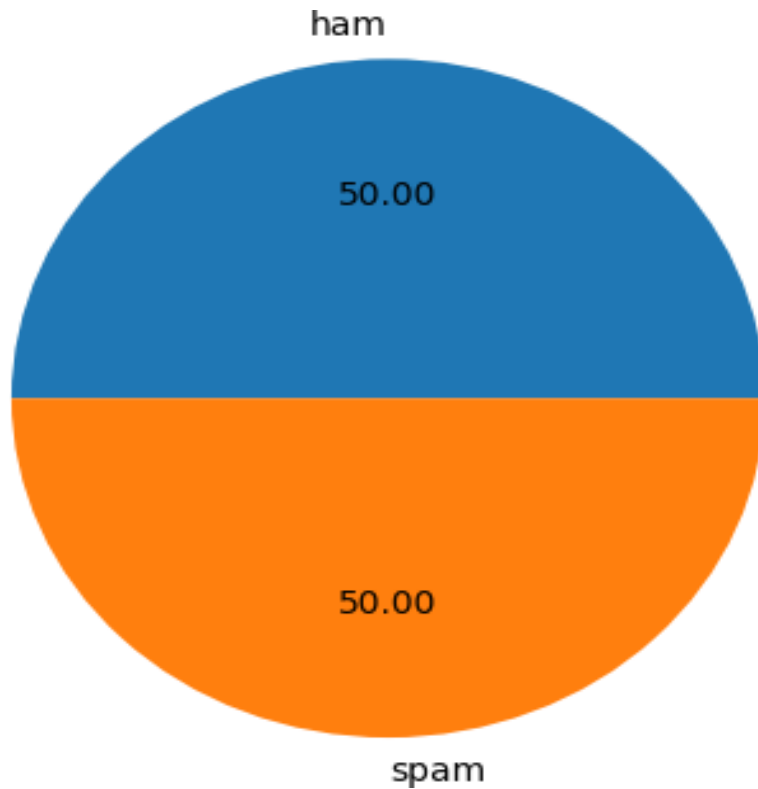
- To enhance model performance, preprocess textual data using tokenization, stop-word removal, stemming/lemmatization, and TF-IDF feature extraction.
 - To determine the best classifier by analyzing and contrasting several machine learning algorithms, including Random Forest, Support Vector Machine (SVM), Naïve Bayes, and Ensemble Methods.
 - To use feature selection and hyperparameter tuning to maximize model accuracy and minimize false positives and false negatives. Metrics such as accuracy, precision, recall, Z-score, F1-score, ROC curve, and AUC are used to assess model performance.
 - To investigate how feature representation, class imbalance, and dataset size affect the overall detection performance.
- Importance of the Research
- To create an effective and scalable spam detection system that can operate on real-time and real-world datasets.
 - Improves cybersecurity by using early spam detection to stop the spread of malware, phishing emails, and fraudulent content. Decreases the quantity of unwanted and unnecessary messages, increasing productivity and communication effectiveness.
 - Aids in recognizing the essential linguistic patterns and message elements that set spam apart from authentic communication. Helps create intelligent email management systems that make use of machine learning and natural language processing. reduces the risk of data security and monetary loss brought on by cyberattacks contained in spam emails.

The use of deep learning and machine learning models for spam detection has already been studied by a number of researchers. After applying multiple machine learning algorithms, Sokolov [8] et al. used the naive Bayes algorithm and achieved an accuracy of 98.07%. Shrivastava [14] et al. used an ensemble model and obtained an accuracy of 97.25%. Using SVM, Laowittayawong [9] et al. obtained an accuracy of 83.33%. Nilam Nur Amir Sjarif [15] et al. used a random forest classifier in conjunction with the term frequency-inverse document frequency (TF-IDF) approach, and the accuracy was 97.5%. M. Rubin Julis [16] et al. achieved 98% accuracy with a support vector machine by utilising a range of machine learning classifiers.

A: Dataset

Several email datasets from well-known online repositories, such as Kaggle and Scikit-learn, as well as proprietary datasets selected specifically for this study, are used in this investigation. The main training dataset, "spam.csv" from Kaggle, consists of 10,000 instances with two attributes: 5,000 are classified as ham (non-spam) and 5,000 are classified as spam. Other datasets are used for testing and evaluating the model; they are in plain text format

and include 574, 1,001, and 956 email samples, respectively. Together, these varied datasets enable the spam detection model to be rigorously trained and thoroughly validated.



Dataset	Ham	Spam	Total
No .of rows	5000	5000	10000

Fig 3: Class distribution in the dataset

III: METHODOLOGY

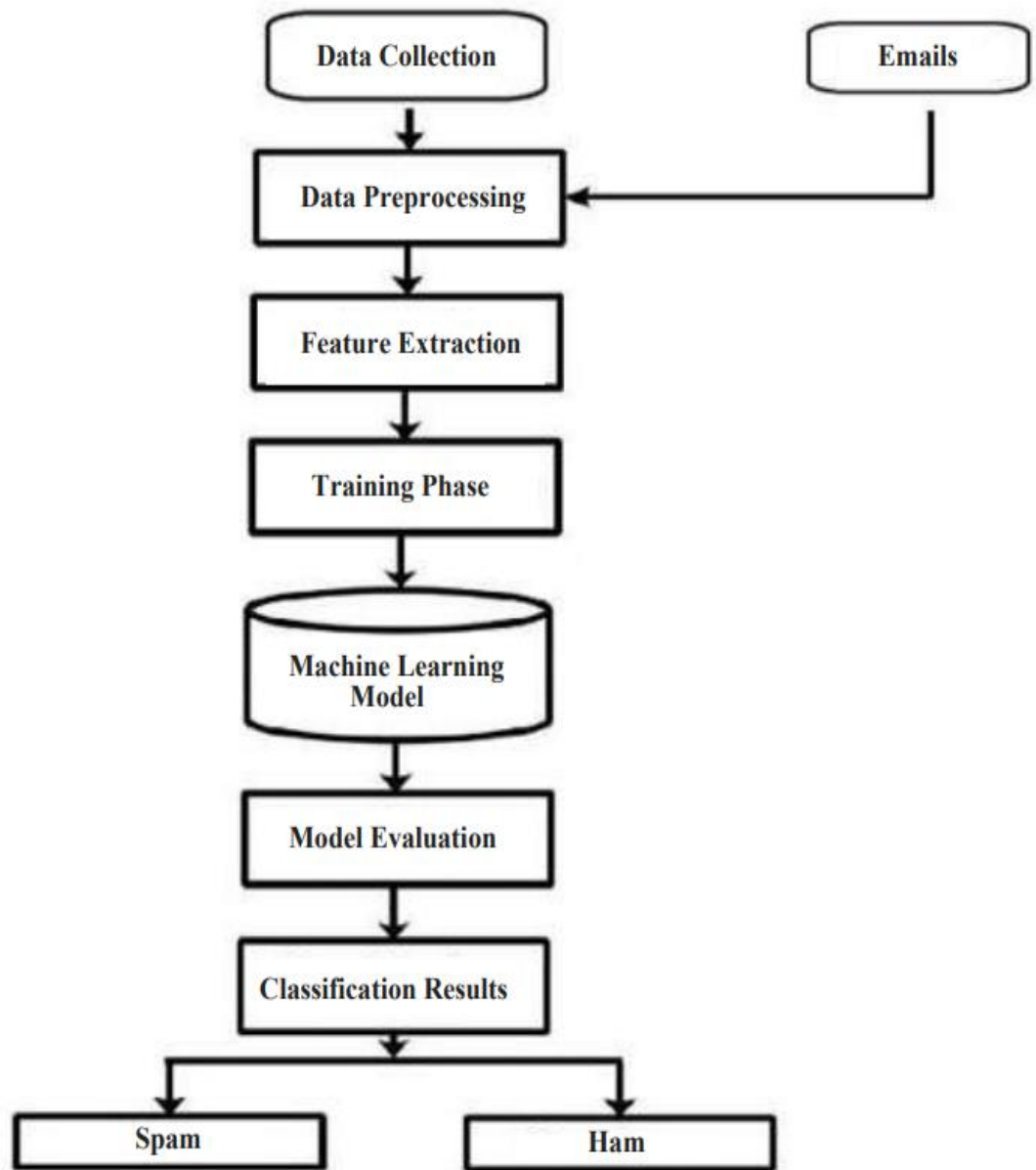


Fig 4: Machine Learning Model Flow Chart

The entire process of a machine learning-based email spam classification system is depicted in this flowchart. Gathering raw email data from an email corpus is the first step in the process. Both spam and non-spam messages are included in this dataset. Preprocessing is the next stage, which gets the emails ready for

examination. Stop words, lowercase text conversion, tokenisation, stemming, and deleting unnecessary characters are a few examples of preprocessing. By following these procedures, you can be sure that the data is clean and ready for feature extraction.

Following preprocessing, the system uses feature extraction to transform email text into useful numerical representations. Word embeddings, bag-of-words, and TF-IDF are examples of common methods. The learning algorithm uses these features as inputs. A machine learning model that learns patterns to differentiate spam from non-spam emails is fed the extracted features during the training phase. Traditional algorithms or a neural network classifier, as demonstrated here, which processes the features through interconnected layers to find intricate relationships within the data, may be included in the model.

Ultimately, each incoming email is classified as either spam or non-spam by the trained model. This automated procedure improves the accuracy of email filtering and assists users in avoiding unsolicited or harmful messages.

A. Data preprocessing

Importing a dataset from websites like Google, Sklearn, Kaggle, or proprietary sources is known as data preprocessing. Inconsistencies, irregularities, missing values, duplicate entries, and NULL values are common in raw data, which can make preprocessing difficult. These procedures are required to guarantee that the dataset is clean and appropriate

for using any chosen algorithm to achieve higher model accuracy. Data cleaning, such as eliminating null values from numerical data, is usually part of the first stage. Additional preprocessing methods, including deleting HTML tags, deleting URLs, converting to lowercase, deleting punctuation, and filtering out stop words, are used for text data. When combined, these text preprocessing methods enhance the dataset's quality and facilitate more efficient model training and prediction.

1. Lowercase
2. Remove HTML Tag
3. Remove URLs
4. Remove stop words
5. Tokenisation
6. Remove Punctuation

1. Lowercase :

In NLP, lowercasing reduces the vocabulary size and ensures uniformity by converting all text characters to lowercase, which enhances model performance and consistency. Particularly in small datasets or search applications, it helps avoid problems brought on by case differences and streamlines later preprocessing steps.

2. Remove HTML Tag:

One popular preprocessing method is to eliminate HTML tags and special characters from text input. By removing superfluous formatting and symbols from the data, this step improves the accuracy and efficacy of the model. Only relevant textual content is used for training and prediction thanks to this preprocessing.

3. Remove URLs:

In natural language processing (NLP), removing URLs from text data is a common preprocessing step to get rid of links that don't add any useful information. By concentrating on the actual content, this cleaning raises the standard of the text input and increases model accuracy.

4. Remove stop words :

Common English words like "is," "and," and "the" are known as stop words because they don't significantly advance a sentence's meaning. They help to simplify the data and increase processing efficiency because they can be eliminated from text data without affecting the overall meaning. For instance, the words "is," "to," and "the" in the sentence "She is going to the market" are stop words that can be omitted without affecting the main idea.

5. Tokenisation:

The process of decomposing a sentence into words or characters and turning them into an array is known as tokenisation. In natural language processing (NLP) tasks, it is a crucial text preprocessing step that breaks down unstructured text into digestible chunks for additional analysis.

6. Remove Punctuation:

Periods, commas, question marks, and exclamation points are examples of punctuation, which is used to help structure and make sense of writing. They make text easier to read and comprehend by delineating sentence boundaries, separating ideas, and indicating pauses or emphasis.

B. Word Embedding Techniques :

1. Tf-Idf

One method for turning text into numerical data is called TF-IDF (Term Frequency–Inverse Document Frequency). It combines two metrics: Inverse Document Frequency (IDF), which lessens the weight of common words that occur in a large number of documents, and Term Frequency (TF), which counts the frequency of a word's occurrence in a document. By highlighting significant words and reducing the influence of commonly used ones, TF-IDF offers a more meaningful representation than Bag of Words or Count Vectorizer.

2. Bag of words :

An NLP text representation method that turns text into numerical features is called the Bag of Words (BoW) model. By representing each sentence as a vector with values indicating either the presence (e.g., 0 or 1) or the frequency of words, it creates a vocabulary of distinct words from the dataset. This procedure creates a matrix where each column represents a distinct word from the dataset.

Top 50 words used in spam:

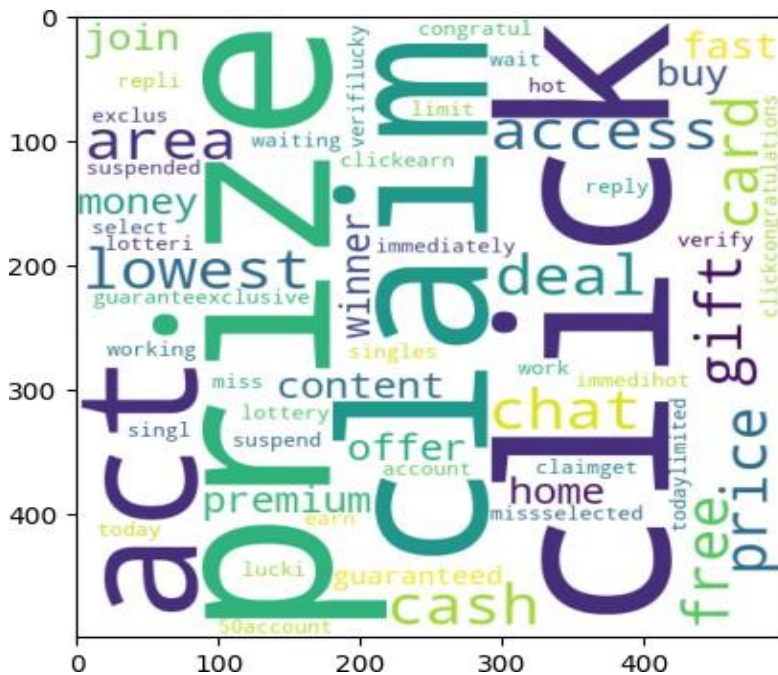


Fig 5: Spam words

Top 50 words used in ham

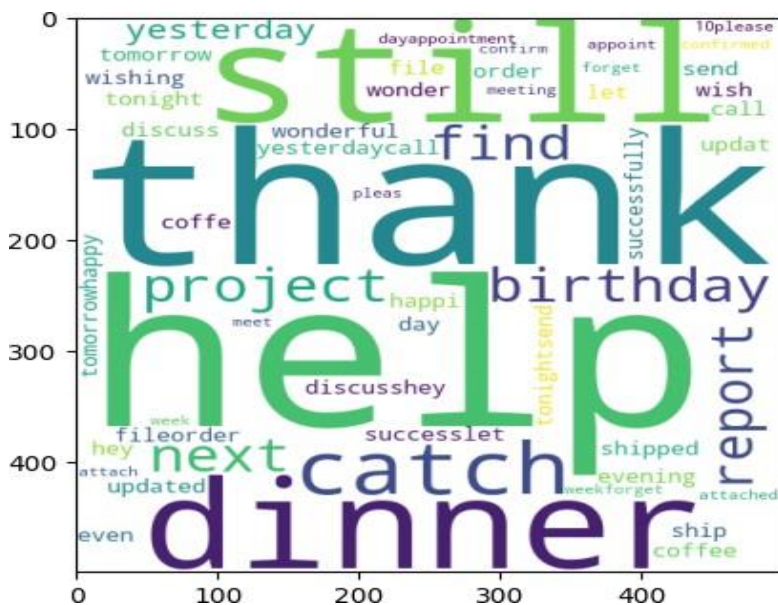


Fig 6: Ham words

Compare how character, word, and sentence counts differ between spam and ham messages

Num_character: Compared to Category 0 (blue), Category 1 (orange) typically has a slightly higher mean and a wider distribution (more characters per entry).

num_words: In comparison to Category 0 (blue), Category 1 (orange) also exhibits a slight tendency toward a higher word count. There is substantial overlap between the distributions.

Number of Sentences: Here, the division is more obvious. While Category 1 (orange) has a distribution centred around two or three sentences, Category 0 (blue) primarily has one sentence (with a very sharp peak at 1).

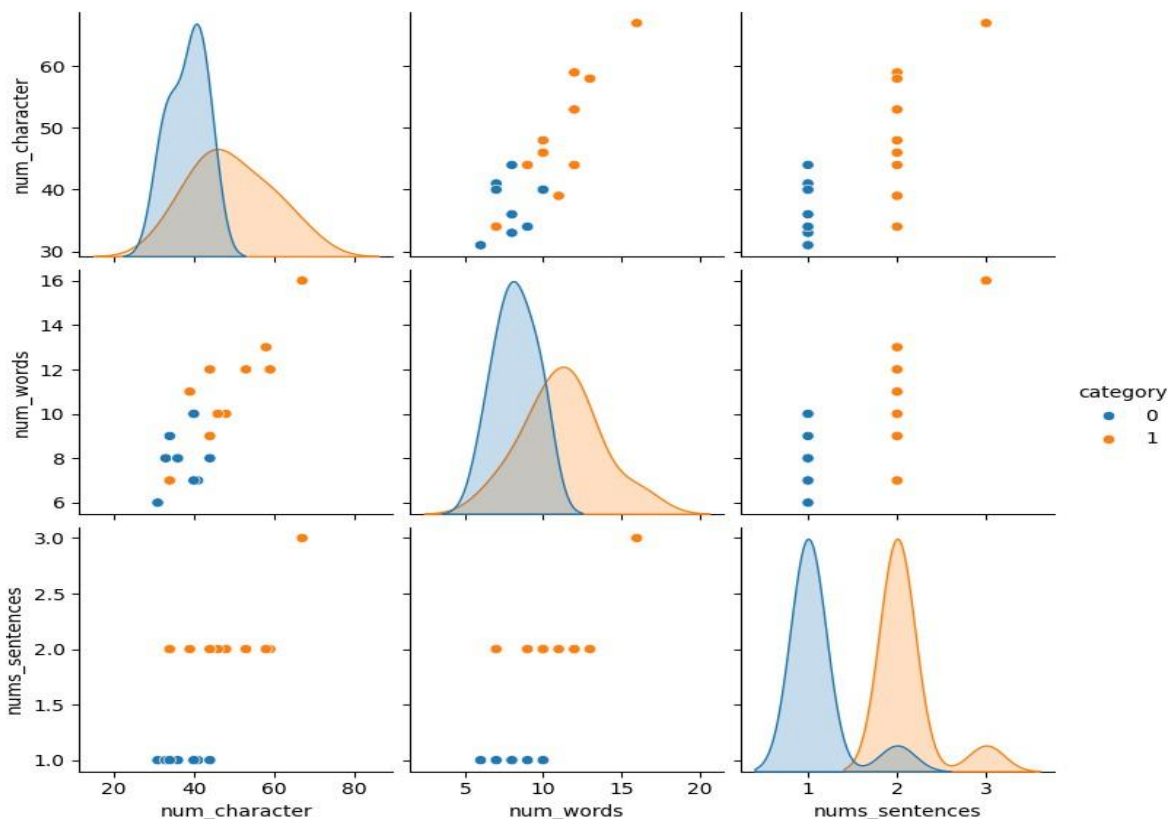


Fig 7: Compare all relations between character, word and Sentence

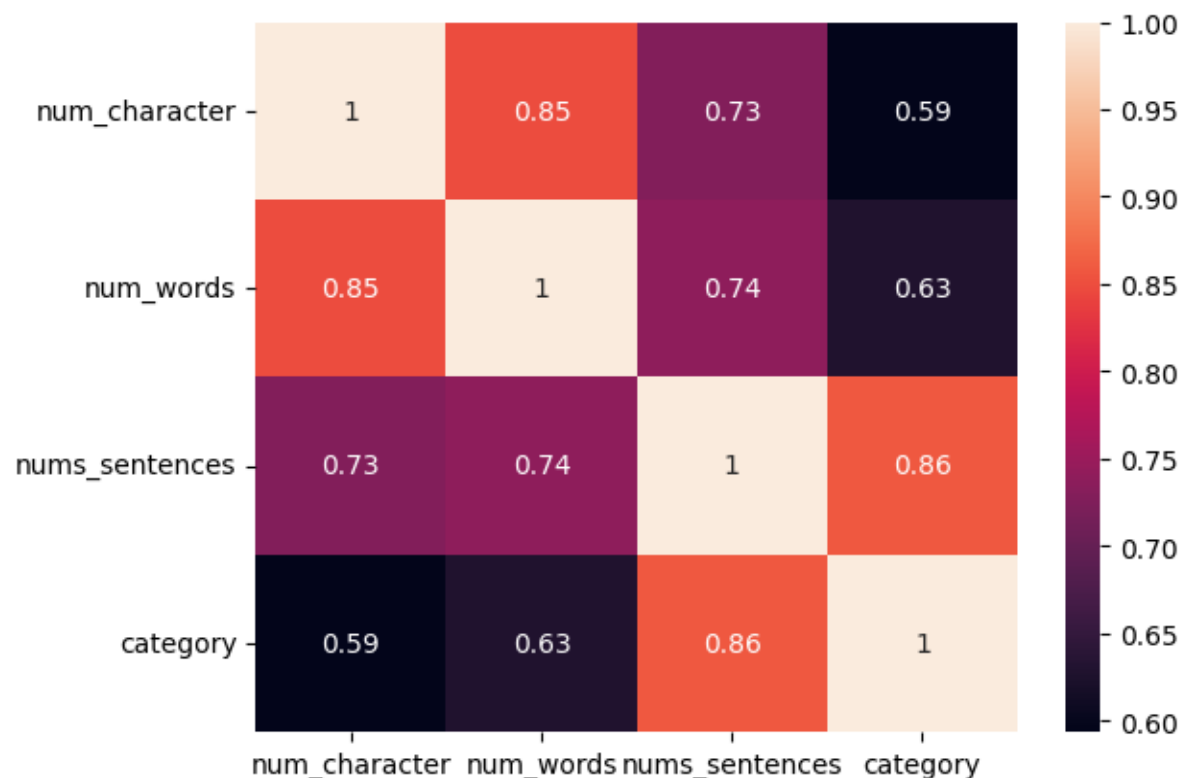


Fig 8: Relationship between character, words, and sentences using Heat Map

The correlation between four text-related features—num_character, num_words, num_sentences, and category—is displayed in Figure 7. A strong positive relationship is indicated by correlation values that are closer to 1, which range from -1 to 1. Character count, word count, and sentence count are the first three features that exhibit strong correlations with one another. For instance, there is a 0.85 correlation between num_character and num_words, indicating that texts with more characters also typically contain more words. Similarly, as would be expected given that longer texts typically contain more sentences, nums_sentences has a strong correlation with both num_words (0.74) and num_characters (0.73).

Additionally, there are significant correlations between the category variable and the textual features. At 0.86, it has the strongest correlation with `nums_sentences`, indicating that the number of sentences in a text has a significant impact on the category to which it is assigned. Its moderately strong correlations with `num_words` (0.63) and `num_characters` (0.59) suggest that longer texts are also more likely to fall into particular categories. The heatmap shows that all features are positively correlated overall. The category seems to be most strongly influenced by the number of sentences, while the strongest correlations are found between the three length-based features.

C. Machine Learning Techniques:

Following the application of various data preprocessing, natural language processing, and word embedding techniques like Bag of Words (BoW) and TF-IDF, the text was converted into real-valued vectors. These vectors were then subjected to analysis using various machine learning algorithms.

- **SVC (Support Vector Classifier):** Uses kernel tricks to handle non-linear boundaries and finds the best hyperplane to divide classes. It works well in high-dimensional spaces.
- **K Neighbours Classifier:** It is straightforward and non-parametric, but it can be slow when dealing with big datasets. It classifies data points according to the majority label among the closest data points.
- **Multinomial NB:** Assuming feature independence, the Naive Bayes model for discrete features, which is frequently employed in text classification, generates quick probabilistic predictions.
- **Decision Tree Classifier:** Creates a tree-like model that can handle both numerical and categorical data, splits on feature values, and is easy to understand.

- **Ada BoostClassifier:** Increases overall accuracy by concentrating on incorrectly classified samples in each subsequent round, combining several weak learners into a strong one.
- **Random Forest:** An ensemble model called a random forest uses arbitrary subsets of data and features to construct numerous decision trees. It reduces overfitting and increases accuracy by combining their predictions.
- **Boosting Gradients:** Decision trees are sequentially trained using gradient boosting, with each new tree fixing the mistakes of the ones before it. Gradient-descent-like optimization is used to minimize loss.
- **Regression Logistic:** A sigmoid function is used to estimate the probability of a class in logistic regression, a linear model for binary classification. It works well with linearly separable data and is easy to understand.
- **Classifier for Extra Trees:** (Highly Randomized Trees): By selecting random feature splits, Extra Trees creates several highly randomized decision trees. This high degree of randomness frequently increases speed and lowers variance.
- **Classifier for Bagging:** Several base models are trained by a bagging classifier using various bootstrapped subsets of the data. To increase stability and lessen overfitting, it averages or votes among these models.

- **XGB Classifier:** Extreme Gradient Boosting is renowned for its high accuracy and quick performance on big datasets. It is highly optimised with regularisation and effective handling of missing data.

IV. EXPERIMENTATION AND RESULTS

Using performance metrics like accuracy and precision, the model analyses several algorithms to determine their overall efficacy. The best-performing model is determined by comparing the accuracy and precision rates of each algorithm. Based on the maximum accuracy or precision attained, the chosen algorithm is then used to predict and categorise emails as either spam or not.

A. Evaluation Metrics:

Accuracy, precision, Recall, F1 score, and Z-score are the evaluation metrics that have been used to compare classifier performance.

Precision :

$$\frac{TruePositives}{TruePositives + FalsePositives}$$

where,

FP stands for False Positive, FN for False Negative, TP for True Positive, and TN for True Negative.

Precision:

Precision quantifies the frequency with which the model's optimistic forecasts come true. By dividing true positives by all predicted positives, it focuses on the calibre of positive predictions.

Accuracy:

Accuracy = $\frac{\text{True Positives} + \text{True Negatives}}{\text{True Positives} + \text{True Negatives} + \text{False Positives} + \text{False Negatives}}$

Machine learning accuracy measures how often a model predicts correctly out of all its predictions. It is calculated as a percentage of correct predictions to total predictions. This simple metric can be misleading on imbalanced class datasets, where precision, recall, and F1-score are more reliable.

Recall:

$$\frac{TruePositives}{TruePositives+FalseNegatives}$$

The number of real positive cases that the model accurately detects is known as recall. It is crucial in high-risk applications because it concentrates on reducing false negatives. A high recall indicates that positive cases are rarely missed by the model. It helps with fraud detection, rare event prediction, and medical diagnosis.

F1 -Score:

$$F1 = 2 * (Precision * Recall) / (Precision + Recall)$$

The F1-score is a crucial metric in classification, serving as the harmonic mean of Precision and Recall. It is used to measure a model's balance between these two important performance indicators.

Handling Imbalance: F1 is especially useful when the dataset is imbalanced, where simple accuracy can become misleading.

Performance Indicator: A high F1-score signifies that the model performs well in both detecting positives (high Recall) and avoiding false alarms (high Precision).

Z-Score :

A Z-score shows the number of standard deviations that a value deviates from the mean.

$Z = (x - \mu) / \sigma$ is used to calculate it.

Features are scaled to mean 0 and standard deviation 1 using Z-score standardisation

It improves the performance of algorithms such as KNN, SVM, and neural networks.

Outliers are also identified by Z-scores; values with $|Z| > 3$ are typically considered outliers.

Consequently, it is employed for anomaly detection as well as preprocessing.

ROC (Receiver Operating Characteristic):

A graph that displays model performance across all classification thresholds is called a ROC curve. The True Positive Rate (Recall) is plotted against the False Positive Rate. The curve illustrates the trade-off between preventing false alarms and detecting positives. Better is a model with a curve nearer the upper-left corner. ROC facilitates model comparisons irrespective of the threshold chosen. It is frequently employed in the assessment of binary classification.

AUC(Area Under the Curve):

AUC summarises model performance by calculating the total area under the ROC curve. It shows how well a model is able to give positive cases a higher ranking than negative cases. The

range of AUC values is 1.0 (perfect classifier) to 0.5 (random). Better class separation ability is indicated by a higher AUC. In contrast to accuracy, it is threshold-independent. AUC is particularly helpful for datasets that are unbalanced.

B: RESULTS AND DISCUSSION

The dataset used in this model is split into two parts: 20% for testing and 80% for training. The accuracy and precision of ten algorithms—including Naive Bayes, Random Forest, Decision Tree, XG Boost, AdaBoost, Gradient Tree Classifier, and Logistic Regression—are assessed. To determine whether an email is spam or not, the model chooses the algorithm with the highest accuracy rate, which can reach 99.20%.

```
svc = SVC(kernel='sigmoid', gamma=1.0)
knc = KNeighborsClassifier()
mnb = MultinomialNB()
dtc = DecisionTreeClassifier(max_depth=5)
lrc = LogisticRegression(solver='liblinear', penalty='l1')
rfc = RandomForestClassifier(n_estimators=50, random_state=2)
abc = AdaBoostClassifier(n_estimators=50, random_state=2)
bc = BaggingClassifier(n_estimators=50, random_state=2)
etc = ExtraTreesClassifier(n_estimators=50, random_state=2)
gbdt = GradientBoostingClassifier(n_estimators=50, random_state=2)
xgb = XGBClassifier(n_estimators=50, random_state=2)
```

Fig 9: Pseudocode of different Machine Learning Algorithms

The sigmoid kernel type and kernel coefficient, $\gamma = 1.0$, were used to train the SVM classifier. $\alpha = 0.2$, an additive

The smoothing parameter was used to train the MNB classifier. `Mix_depth = 5`, the smallest number of samples needed to split an internal node, and `random_state = 2`, the estimator's randomness, were used to train the DT classifier. The algorithm to be used in the optimisation problem, `solver = liblinear`, and the penalty norm, `penalty = "l1"`, were used to train the LR classifier. Finally, the randomness of bootstrapping of samples used to build trees (`random_state = 2`) and the number of trees in the forest (`n_estimators = 50`) were used to train the RF classifier.

MNB had the highest accuracy of any machine learning algorithm, at 97.13%. The MNB classifier's precision and accuracy are both high, indicating fewer false positives and false negatives, and its accuracy for email spam detection is 97.13%, other check ROC graphs to analysis of value Additionally, the Multinomial Naive Bayes classifier has the highest score, meaning it makes more accurate predictions about a particular class. Display every detail in Figures 10 and Table 1.

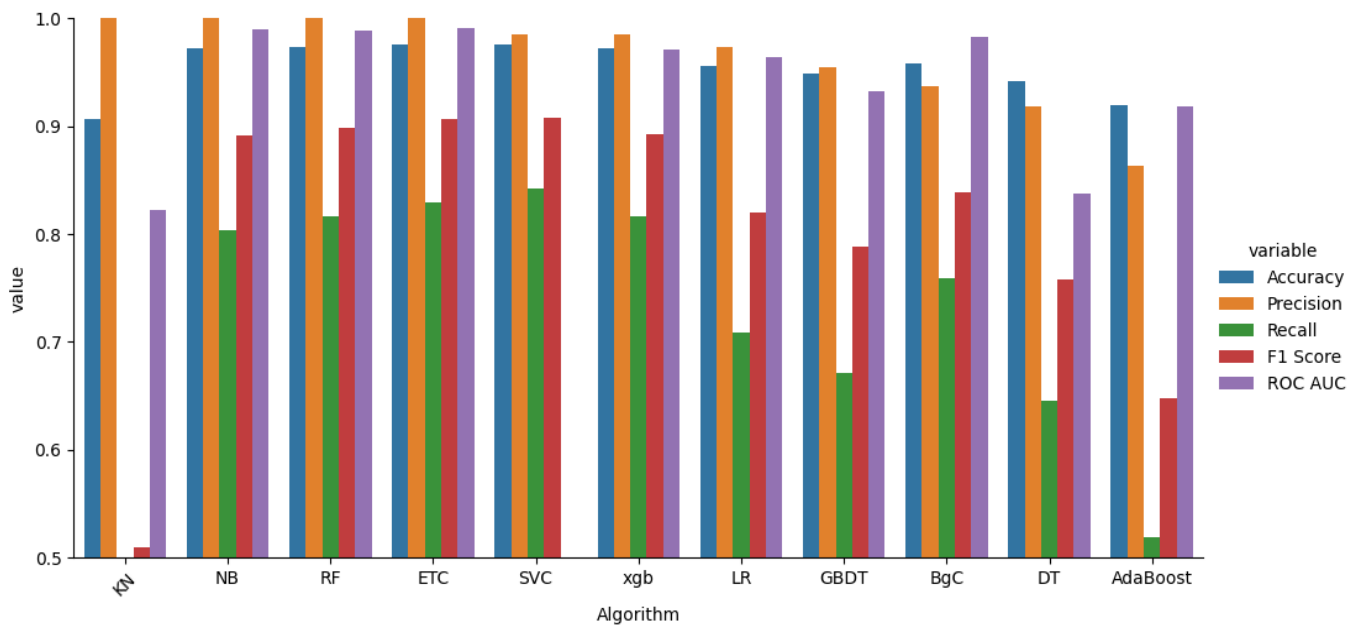


Fig 10: Compare different Machine Learning Algorithms

	Algorithm	Accuracy	Precision	Recall	F1 Score	ROC AUC
1	KN	0.906726	1.000000	0.341772	0.509434	0.822514
2	NB	0.972197	1.000000	0.803797	0.891228	0.990053
5	RF	0.973991	1.000000	0.816456	0.898955	0.989035
8	ETC	0.975785	1.000000	0.829114	0.906574	0.990529
0	SVC	0.975785	0.985185	0.841772	0.907850	NaN
10	xgb	0.972197	0.984733	0.816456	0.892734	0.971314
4	LR	0.956054	0.973913	0.708861	0.820513	0.963672
9	GBDT	0.948879	0.954955	0.670886	0.788104	0.932294
7	BgC	0.958744	0.937500	0.759494	0.839161	0.983017
3	DT	0.941704	0.918919	0.645570	0.758364	0.837999
6	AdaBoost	0.920179	0.863158	0.518987	0.648221	0.918532

Table No. 1: Algorithm comparator Accuracy, Precision, Recall, F1 Score and ROC AUC

C: Compare all Naïve Bayes

I. Gaussian Naïve Bayes

II. Multinomial Naïve Bayes

III. Bernoulli Naïve Bayes

Gaussian Naïve Bayes:

1. ROC Curve

The trade-off between (TPR) and(FPR) at different classification thresholds is represented visually by the ROC curve. The model starts by accurately identifying positives with few false alarms, as evidenced by the sharp rise in your curve. This form demonstrates how well the classifier separates positive and negative classes and maintains good sensitivity across thresholds.

2. ROC AUC

Strong discriminative power is indicated by the ROC AUC value of 0.8723. The likelihood that the model will rank a positive sample higher than a negative one is represented by the AUC.

Your model outperforms random guessing and exhibits dependable class separation with an AUC of 0.9.

3. Precision

With an accuracy of 88.52%, the model is accurate in almost nine out of ten predictions. Although this is a good overall score, other metrics help paint a more complete picture of performance because accuracy by itself can be deceptive in unbalanced datasets.

4. Matrix of Confusion

852 TN, 135 TP, 105 FP, and 23 FN are displayed in the confusion matrix. The model rarely overlooks positive cases, as evidenced by the low number of false negatives. A greater number of false positives, however, indicates that some negative samples may have been mistakenly labelled as positive.

5. F1 Score, Precision, and Recall

Just slightly more than half of the predicted positives are accurate, with a precision of 56.25%. At 85.44%, recall is high, demonstrating a strong ability to identify true positives. The F1 score of 0.678 balances both and reflects moderate overall performance.

6. Z-Scores

The first five Z-scores (≈ 0.52) indicate the predicted probabilities fall slightly below the mean confidence level. This helps understand how each prediction deviates from the model's average output.

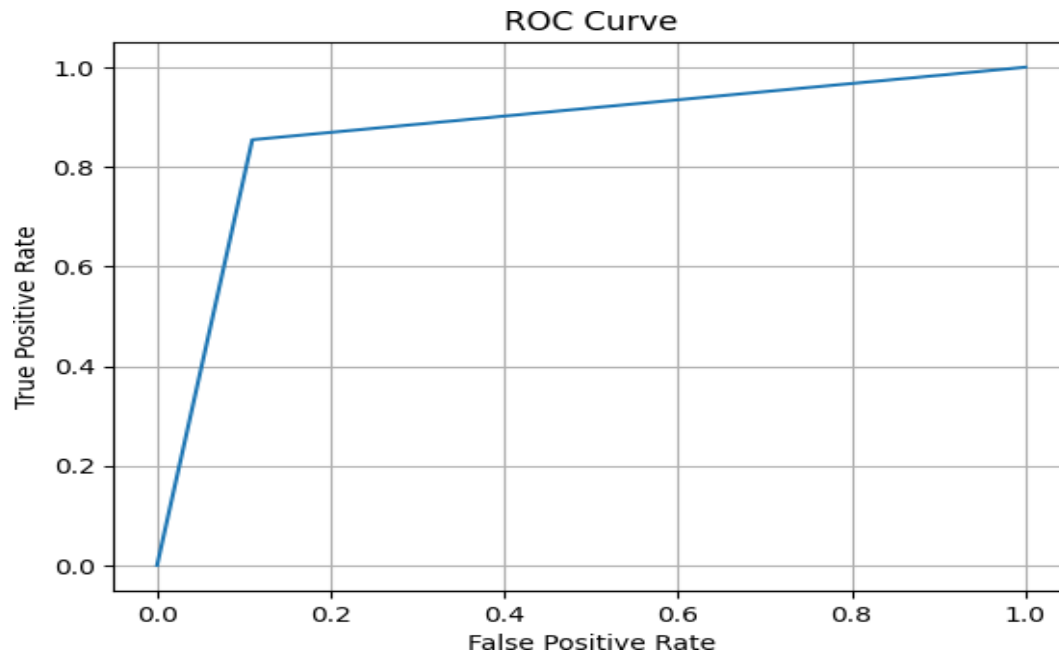


Fig 11: Analysis curve of Gaussian naïve Bayes

Multinomial Naïve Bayes:

Analysis of Model Performance Metrics

Based on the provided ROC Curve image and the various performance metrics, here is an analysis of the model's performance on this classification task.

Key Performance Metrics

The model demonstrates strong performance across several metrics, indicating a high level of classification accuracy, though with a notable trade-off in identifying all positive instances.

Accuracy: 0.972 (97.2%). This is very high, suggesting that the model correctly predicts the outcome (both positive and negative) for the vast majority of samples.

Confusion Matrix: 957 & 0 \ 31 & 127

TN: 957 (Correctly classified negative instances).

False Positives (FP): 0 (Negative instances incorrectly classified as positive (FN): 31 (Positive instances incorrectly classified as negative). True Positives (TP): 127 (Correctly classified positive instances).

The zero False Positives (FP=0) is a remarkable result, indicating the model is perfect at identifying negative cases and never makes a Type I error.

Precision: 1.0 (100%).

Recall (True Positive Rate): 0.804 (80.4%)

This is the fraction of actual positive cases that were correctly identified. The value of 0.804 means that the model misses about 20% of the true positive cases (the 31 False Negatives). This represents the primary weakness of the model.

F1 Score: 0.891

The harmonic mean of Precision and Recall. This score is high, but the fact that it is lower than the Accuracy and Precision is largely driven down by the imperfect Recall score. ROC AUC 0.990 and 0.872

There is a discrepancy in the two reported ROC AUC values. 0.990: This is an exceptionally high value, suggesting that the model's probability scores are highly effective at distinguishing

between the two classes across all classification thresholds. 0.872: This is also a strong score, but lower.

ROC Curve Analysis (Image):

The curve shows a very sharp rise from (0, 0) to roughly (0.1, 0.85) and then maintains a gradual slope towards (1.0, 1.0). This shape is consistent with a strong classifier. It achieves a very high True Positive Rate (Recall) with only a small increase in the False Positive Rate, which aligns better with the 0.990 AUC score for distinguishing probabilities across thresholds than the 0.872.

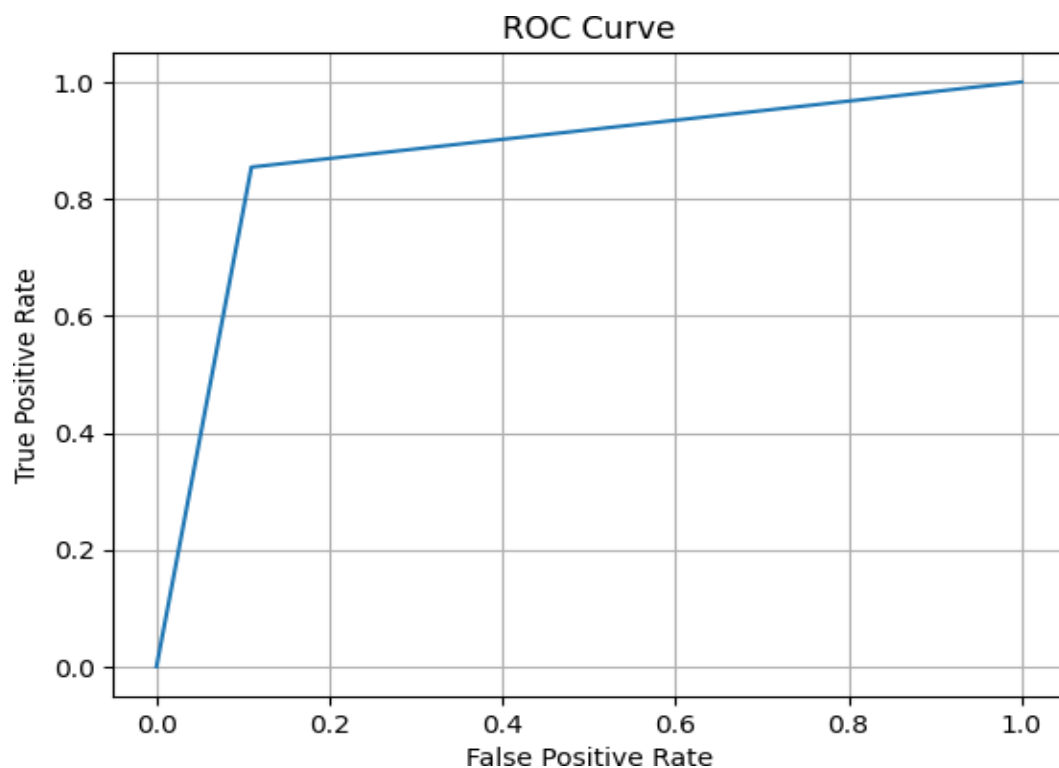


Fig 12: Analysis curve of multinomial Naïve Bayes

Bernoulli Naïve Bayes

Analysis of the ROC Curve

The provided graph is a Receiver Operating Characteristic (ROC) Curve, which is a vital tool for evaluating the performance of a binary classification model across all possible classification thresholds.

False Positive Rate (X-axis)

The False Positive Rate (FPR) is plotted on the X-axis and is often called the 1-Specificity. It represents the proportion of all actual negative cases that were incorrectly predicted as positive (Type I error).
$$FPR = \frac{\text{False Positives}}{\text{False Positives} + \text{True Negatives}}$$
A perfect model would have an FPR of 0.0, meaning it never issues a false alarm.

Interpretation of the Curve

The curve shows the trade-off between TPR and FPR as the decision threshold changes. The model represented by this curve is an excellent classifier. It rises steeply, reaching a high TPR (close to 1.0) very quickly while maintaining a very low FPR (close to 0.0). The closer the curve is to the top-left corner (0, 1), the better the model is at separating the classes. The Area Under the Curve (AUC) for this plot would be very high, likely above 0.95, indicating that the model is highly effective and performs much better than a random classifier.

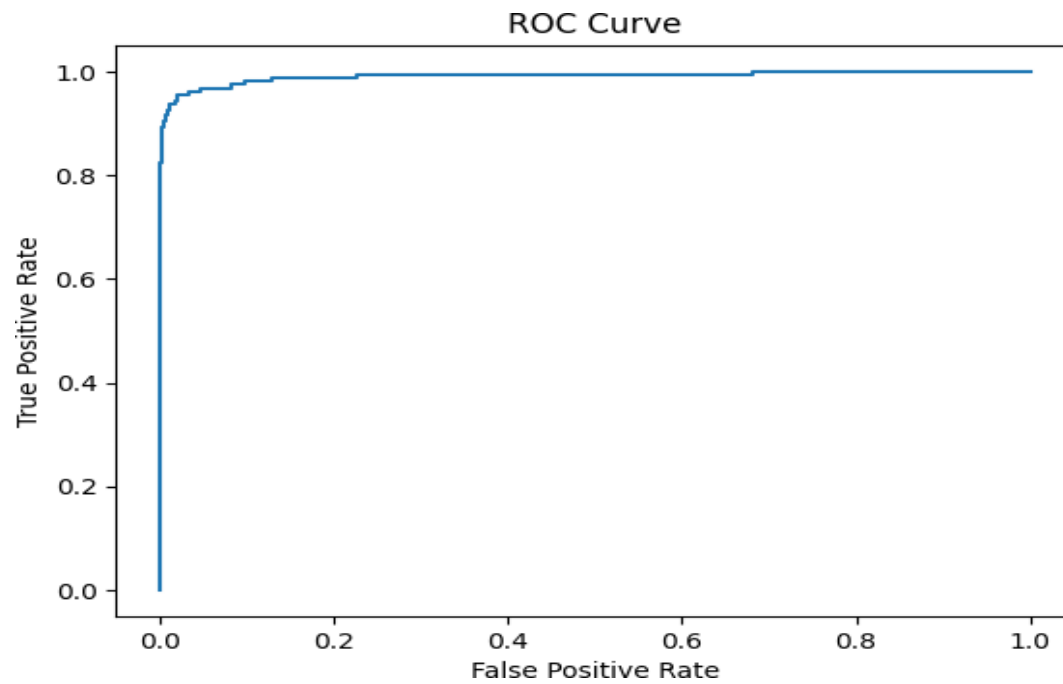
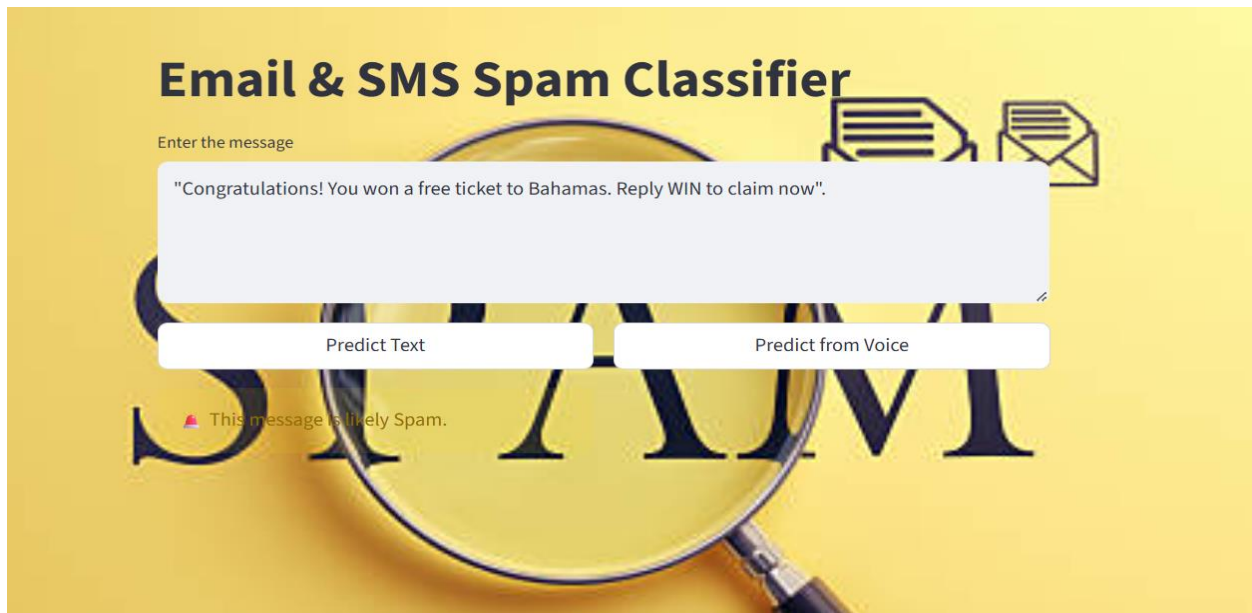


Fig 13: Bernoulli Naïve bay

D: FINAL RESULT AND OUTPUT

OUTPUT FOR SPAM MESSAGE :



The screenshot shows a web application titled "Email & SMS Spam Classifier". It has a yellow background with a magnifying glass over the word "SPAM". The interface includes a text input field with the message "Congratulations! You won a free ticket to Bahamas. Reply WIN to claim now". Below the input are two buttons: "Predict Text" and "Predict from Voice". The output shows a red warning icon and the text "This message is likely Spam."

Email & SMS Spam Classifier

Enter the message

"Congratulations! You won a free ticket to Bahamas. Reply WIN to claim now".

Predict Text Predict from Voice

🚨 This message is likely Spam.

OUTPUT FOR HAM MESSAGE :



The screenshot shows the same web application as above, but with a different message in the input field: "Are you coming to the meeting tomorrow?". The output shows a green checkmark icon and the text "This message seems Not Spam."

Email & SMS Spam Classifier

Enter the message

"Are you coming to the meeting tomorrow?"

Predict Text Predict from Voice

✅ This message seems Not Spam.

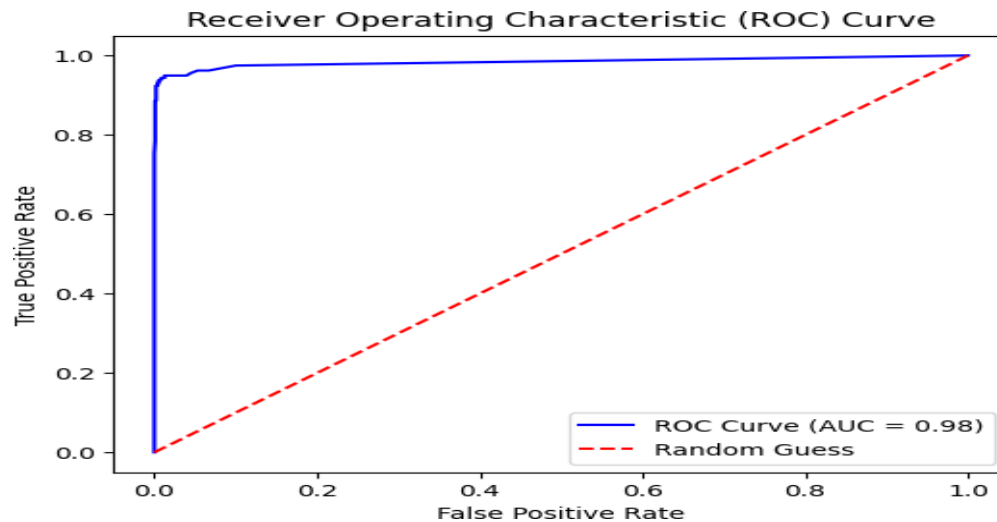


Fig. 14: Final Output in the form of ROC

All algorithms are used with an accuracy rate of 98.5%.98.6% precision means that there aren't many false positives when it comes to spam detection.

90.5% recall indicates that real spam was successfully detected, but 9.5% was missed.

F1 Score: 94.4%, demonstrating a strong overall detection by striking a balance between precision and recall.

With an AUC of 0.98, the ROC curve further validates the model's quality by demonstrating its high ability to differentiate between spam and ham messages across thresholds. Very high true positive rates with few false positives are shown by the ROC curve near the top-left corner

V: Comparison with Previous Work

A thorough comparison of the accuracy of the machine learning models suggested in this study with those documented in previous studies on email spam detection is shown in Figs. 8 and 9. According to earlier research, the accuracy of the Naive Bayes (NB) classifier was 98.07%, that of the Ensemble Model (EM) was 97.25%, and that of the SVM model was 86.33%. The text data in this study was thoroughly preprocessed before features were extracted using the Count Vectorizer and TF-IDF techniques. Ten distinct algorithms—including several Naive Bayes variants like Gaussian NB, Multinomial NB, and Bernoulli NB—were tested in order to bolster the analysis and fully assess their performance. To find the best model, each algorithm was thoroughly examined. Compared to earlier research, this thorough analysis offers more trustworthy insights and strengthens the foundation. Accuracy, precision, recall, F1-score, Z-score, and other metrics were used in the full model evaluation to make sure the final prediction model is reliable, effective, and able to handle real-world spam detection scenarios.

VI. CONCLUSIONS:

The comparative analysis proves the efficacy of the suggested machine learning models for email spam detection, which make use of thorough data preprocessing and word embedding strategies. The models—which included SVM, MNB, DT, LR, and RF—achieved high accuracy rates, either matching or outperforming the findings of earlier research. This suggests that a variety of classification algorithms combined with strong preprocessing procedures can greatly improve spam detection performance, which makes these methods promising for practical email filtering applications. The SVM model is the most reliable for spam detection among these due to its balanced precision and

recall. Other models, like MNB and LR, work well, but they have drawbacks in some situations.

All things considered, choosing the right model based on particular classification requirements is essential for achieving the best results.

Classifying spam emails is important for classifying emails and differentiating between spam and non-spam emails. Large bodies can use this technique to distinguish between good emails and just the emails they want to receive.

REFERENCES

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [2] A. Géron, *Hands-On Machine Learning with Scikit- Learn, Keras, and TensorFlow*, 2nd ed. O'Reilly Media, 2019.
- [3] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. Pearson, 2023.
- [4] M. Kuhn and K. Johnson, *Applied Predictive Modelling*. Springer, 2013.
- [5] C. D. Manning and H. Schütze, *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.

[6] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.

[7] S. Vajjala, B. Majumder, A. Gupta, and H. Surana, *Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems*. O'Reilly Media, 2020.

[8] Sokolov, Mark, Kehinde Olufowobi, and Nic Herndon. "Visual spoofing in content-based spam detection." In 13th International Conference on Security of Information and Networks, pp. 1-5. 2020.

[9] Laowittayawong, Harit, and Woratat Makasiranondh. "Spam Mail Filtering Using Information Extraction, Stop-word Removal and Classification Techniques." In Proceedings of 6th International On Networking And Computer Application (2018).

[10] Navaney, Pavas, Gaurav Dubey, and Ajay Rana. "SMS spam filtering using supervised machine learning algorithms." In 2018 8th International Conference on Cloud Computing, Data Science & Engineering (Confluence), pp. 43-48. IEEE, 2018.

[11] Julis, M. Rubin, and S. Alagesan. "Spam Detection In Sms Using Machine Learning through Textmining." International Journal of Scientific & Technology Research 9, no. 02 (2020).

[12] Kaggle. (n.d.). *Kaggle: a machine learning and data science community*. Retrieved October 5, 2025, from <https://www.kaggle.com>.

[13] GeeksforGeeks. n.d. "GeeksforGeeks: A Computer Science and I convery theory part machine learning and NLP " Accessed October 5, 2025. <https://www.geeksforgeeks.org>.

[14] Shrivastava, Akhilesh Kumar, Amit Kumar Dewangan, S. M. Ghosh, and Devendra Singh. "Development of Proposed Ensemble Model for Spam e-mail Classification." *Information Technology and Control* 50, no. 3 (2021).

[15] Sjarif, Nilam Nur Amir, Nurulhuda Firdaus Mohd Azmi, Suriyati Chuprat, Haslina Md Sarkan, Yazriwati Yahya, and Suriani Mohd Sam. "SMS spam message detection using term frequency-inverse document frequency and random forest algorithm." *Procedia Computer Science* 161 (2019): 509-515.

[16] Julis, M. Rubin, and S. Alagesan. "Spam Detection In Sms Using Machine Learning through Textmining." *International Journal of Scientific & Technology Research* 9, no. 02 (2020).

[17] According to global email traffic statistics, the number of emails sent and received per day has risen from 3823 million in 2018 to a projected 4894 million by 2027, reflecting a consistent annual growth trend in worldwide email communication.